CALIGA

Internal Tool for Reader Configurations, Firmware Testing, and Reader Flashing

Purpose, use, setup, and code guide

This document explains what CALIGA is for, how each team uses it, how to install and run it, and how every part of the code works. It's written for someone who has never seen CALIGA before.

Item	Value
Repository	github.com/jhepburn-hue/Internal-Tool-CALIGA
Related repository	github.com/jhepburn-hue/INI-Generator (INI generation rules)
Owner and admin	James H. (jhepburn@wavelynx.com)
Database schema version	Migration 0006_job_files
Document date	October 11, 2026

Contents

Part 1. Overview . . . 5
1.1 What CALIGA is . . . 5
1.2 Key concepts . . . 5
1.3 How the pieces connect . . . 7
1.4 Roles and permissions . . . 8
1.5 Configuration lifecycle . . . 8
Part 2. Using CALIGA . . . 10
2.1 Signing in and moving around . . . 10
2.2 Configurations . . . 10
2.3 Firmware testing (QA and SET) . . . 12
2.4 Failures (SET) . . . 13
2.5 Flashing a reader . . . 14
2.6 Tools . . . 14
2.7 The progress window . . . 15
Part 3. Setup and operations . . . 16
3.1 What you need . . . 16
3.2 Installing . . . 16
3.3 Settings (.env) . . . 16
3.4 Database and migrations . . . 18
3.5 Running CALIGA . . . 19
3.6 Routine maintenance . . . 19
3.7 Tests . . . 19
3.8 Troubleshooting . . . 20
3.9 Before hosting CALIGA for others . . . 20
Part 4. Code guide . . . 21
4.1 Repository layout . . . 21
4.2 How a request flows . . . 22
4.3 Sign-in and permissions (auth.py) . . . 23
4.4 Data model (models.py) . . . 23
4.5 Routes (routes/) . . . 24
4.6 Background job queue (jobs.py) . . . 25
4.7 Services (services/) . . . 26

4.8 Pages and shared JavaScript (templates/) . . . 28
4.9 Browser flasher (static/js/osdp-flasher.js) . . . 29
4.10 Utilities, scripts, and migrations . . . 30
4.11 Tests (tests/) . . . 30
4.12 Conventions to keep . . . 31
4.13 Known limitations and future work . . . 31
Part 5. Reference . . . 32
5.1 Routes . . . 32
5.2 Pinned packages . . . 34
5.3 Database tables . . . 34

How to read this document

The document has five parts. Read the ones you need.

Part	Read it if you want to
1. Overview	Understand what CALIGA does, the words it uses, and how its pieces connect.
2. Using CALIGA	Use the site as Sales, QA, SET, or the admin. Step by step, page by page.
3. Setup and operations	Install CALIGA, configure it, run it, keep it healthy, and fix common problems.
4. Code guide	Change or maintain the code. Every file and every important function is explained.
5. Reference	Look up every route, setting, and database column.

Words in `this font` are names from the code, file paths, settings, or commands you type.

Part 1. Overview

1.1 What CALIGA is

CALIGA is an internal web application for the teams that set up and validate WaveLynx access control readers. A reader's behavior (LED colors, beeper, card technologies, mobile credentials, keys, and much more) comes from a **configuration profile**. Each customer or product line has its own profile. CALIGA keeps every profile in one database and covers the whole life of a profile:

- **Sales** drafts new profiles and edits them while they're proofs of concept.
- **QA** tests profiles against each firmware release, criterion by criterion.
- **SET** (the engineering team) investigates failures, edits the settings file, rebuilds firmware, flashes a physical reader, and asks QA to test again.
- Everyone can download the settings file and built binaries, look up deployment tokens, and use the engineering tools.

CALIGA connects to the systems the teams already use. It builds binaries with Forge, stores files in Google Cloud Storage, opens bug tickets in Jira, posts alerts in Slack, and registers binaries in PocketBase. It flashes readers straight from the browser through a USB RS-485 adapter.

1.2 Key concepts

These words appear throughout CALIGA and this document.

Term	Meaning
Configuration (profile)	One named set of reader settings, such as `CQU1`. Stored as a row in the `configurations` table. Has a status: POC, Active, Production, or Archived.
Config ID	The profile's number. Stored as decimal (`config_id_dec`) and hex (`config_id_hex`). Written into the reader as `cfg_id` in the INI's `[mfg_data]` section.
INI file	The text settings file the firmware build reads. Sections like `[av]` hold keys like `idle_color = "blue"`.
Master INI	The official INI for a configuration and firmware version, kept in GCS at `input/<version>/<config>.ini`.
settings_default.ini	The reader's default settings. INIs list only what differs from these defaults.
Partial INI	A small INI that overrides a few settings on top of another profile.
Profile BIN	A compiled binary of the profile's settings, built by Forge.
Firmware BIN (DCK)	The full firmware image with the profile built in. This is what gets flashed.
Forge	WaveLynx's build service. It turns an INI into a profile or firmware binary. Sits behind Google IAP.
IAP cookie	The browser cookie that proves you're signed in to Forge. CALIGA reuses it to start builds. It expires.

Term	Meaning
GCS	Google Cloud Storage. The `wavelynx_apex_config` bucket holds master INIs, build output, and Forge folders. CALIGA only ever writes inside `forge/jhepburn@wavelynx.com/`.
INI-Generator	A separate WaveLynx package that turns configuration values into an INI. CALIGA uses it when no master INI exists.
PocketBase token	A short code (like `c-0W3u5Qdp`) that identifies an uploaded binary in PocketBase.
Firmware run (FW run)	One round of testing against one firmware version (v5.4.10 or v5.4.11).
Test group	One configuration inside a run. Has a status (Untested, Passed, Failed), a tester, and a Jira key.
Test case (criterion)	One check inside a test group, such as "Idle LED: Red". QA marks each one Passed or Failed.
Revision history	A log of changes to a configuration: edits, status changes, promotions, flashes, re-tests.
OSDP	Open Supervised Device Protocol. How control panels and tools talk to readers over RS-485.
RS-485 adapter	The USB device that connects a computer to a reader's OSDP wires.
Web Serial	A Chrome and Edge feature that lets a web page talk to a USB serial device. CALIGA flashes through it.
CSN	Card Serial Number. Readers can report a card's CSN in several bit formats, set per card technology.
LEAF Si / LEAF Cc	WaveLynx card applications. Cc uses a customer keyset (LkXXXXX).
Keyset	A named set of cryptographic keys, written LkXXXXX. Key slots in the INI refer to them.
Wallet	Mobile wallet credentials. Apple Wallet uses ECP DESFire (Meridian). Android Wallet uses MiFare2Go.
TCI	Terminal ID for wallet transactions. Six hex characters, such as `0209FE`.
Job	Slow work (a Forge build, a PocketBase sync) that runs in the background while the page shows progress.

1.3 How the pieces connect

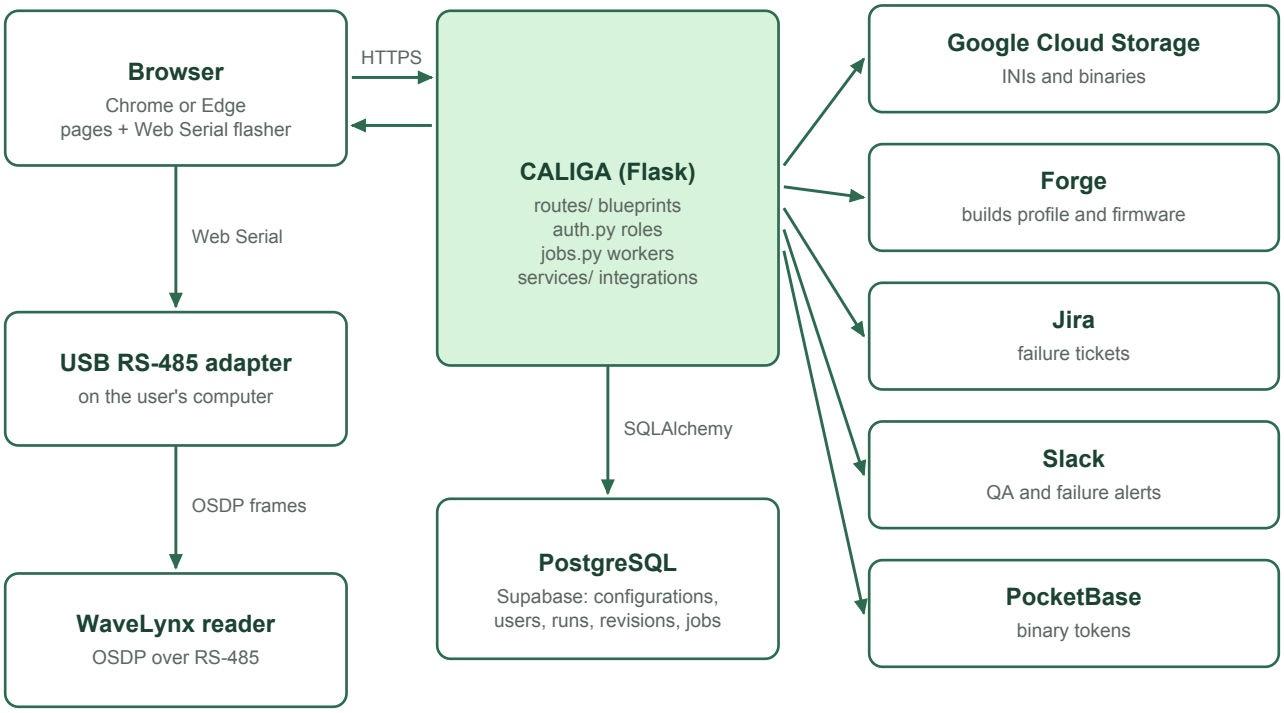


Figure 1. How CALIGA's parts connect.

CALIGA is a Python **Flask** web application. People use it in a browser. The application stores its data in a **PostgreSQL** database hosted by Supabase and reaches the other systems over the network:

System	What CALIGA uses it for
PostgreSQL (Supabase)	Every configuration, user, test run, test case, revision, OEM SKU, background job, and the files jobs produce.
Google Cloud Storage	Reads master INIs and built binaries. Writes generated and edited INIs to the Forge folder. It never writes anywhere else in the bucket.
Forge	Builds profile and firmware binaries from an INI. CALIGA starts the build and waits for the file in GCS.
Jira	Creates an "Internal Bug" when a test group fails, assigns it on claim, closes it on re-test.
Slack	Posts QA test requests, failure alerts, and re-test requests through a webhook.
PocketBase	Looks up and registers profile and firmware binaries under token names.
The reader	Flashed from the browser over Web Serial and OSDP, through the user's own adapter.

1.4 Roles and permissions

Every user has one role. The role decides what they can do. The server checks the role on every action, so hiding a button isn't the only protection.

Action	Sales	QA	SET
View configurations, testing runs, and tools	Yes	Yes	Yes
Create configurations, edit POC configurations	Yes		
Move a configuration between POC and Active, archive it	Yes		
Alert QA about an Active configuration	Yes		Yes
See revision history	Yes		Yes
Start firmware runs, assign test groups, submit results		Yes	Yes
Flash a reader		Yes	Yes
Failures dashboard: claim, edit INI, flash with edited INI, re-test			Yes
Download INIs and binaries, use the tools, look up and sync tokens	Yes	Yes	Yes

Separate from roles, a user can be an **admin**. Admins manage the user list on the **Manage Users** page. James H. is the only admin. Only people in the user list, and marked active, can use CALIGA.

1.5 Configuration lifecycle

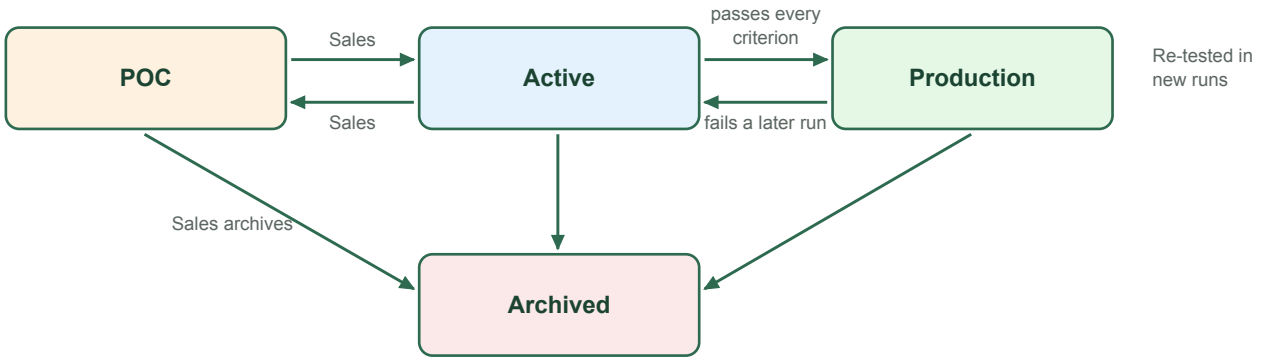


Figure 2. Configuration statuses and who moves them.

Status	Meaning	What's allowed
POC	Proof of concept. A draft that Sales is still shaping.	Editable by Sales. Can't be tested.
Active	Finished and waiting for firmware validation.	Locked against edits. Can be added to firmware runs. Sales or SET can alert QA in Slack.
Production	Passed every criterion in a firmware run. Approved for manufacturing.	Can be re-tested in new runs. A failure sends it back to its previous status (Active or POC).
Archived	Retired. Kept for reference.	No editing, no tools, no downloads, not offered for testing. Only a database change can restore it.

Note. A configuration only reaches Production automatically, by passing a test run. Nobody can set it by hand. A Production configuration that fails a later run is demoted, and the demotion is recorded in its revision history.

Part 2. Using CALIGA

2.1 Signing in and moving around

Open CALIGA in Chrome or Edge (for example `http://127.0.0.1:5000` when it runs on your computer). The top bar has the main areas: **Home**, **Configurations**, **Testing**, **Failures** (SET only), and **Tools**. Your name and role are on the right. Messages about what just happened appear in a colored bar under the top bar. Close one with its X.

Signing in today (development login). Click the user menu on the right and pick your name. A yellow banner reminds you that anyone using this CALIGA can pick any user. That's acceptable on your own computer and not on a shared server. Google sign-in replaces this once CALIGA is hosted (see section 3.9).

- Pages other than Home need a signed-in user. Without one, CALIGA sends you to Home and asks you to pick a user.
- People who aren't in the user list, or who were deactivated, can't sign in.
- **Sign Out** is in the user menu. A sign-in lasts 12 hours by default.

Managing users (admin only)

James H. sees **Manage Users** in the user menu. The page lists everyone with their role and access.

- **Add a user:** enter first name, last name initial, a @wavelynx.com email, and a role, then click **Add User**.
- **Change a role:** pick a new role in the person's row. It saves right away.
- **Deactivate:** click **Deactivate**. The person is signed out on their next click and can't sign in again. **Reactivate** undoes it.
- You can't deactivate yourself, and admin rights can't be changed from the page.

2.2 Configurations

Finding a configuration

Configurations lists every profile in Config ID order. Search by name, hex ID, or decimal ID, and filter by status. Click **View Details** to open one.

The details page

The details page shows every setting, grouped in cards:

Card	Settings
Reader & UI Interface	Idle LED, Credential Report LED, Beeper, Keypad Format, Tamper Monitoring, CASI Output Format, Supervision State
Card Credentials & Prox	Card Type, Facility Code, Starting Badge, Bitstream, FSK Prox, ASK Prox, Prox Filter, Prox Filter Description
Mobile & Wireless Config	BLE Advertising Config, BLE Advertising Name, BLE Functionality, NFC Functionality, Mobile Keyset, Legacy Credentials, Transport Mode
CSN Protocol Configurations	MFC, EV1/EV2, iClass, ISO 15693, ISO 14443A CSN reporting (Off, On, Standard, or a bit format)
LEAF & Wallet	Leaf Si and Leaf Cc applications, Keyset ID, Custom Keyset, ECP DESFire (Apple Wallet), MiFare2Go (Android Wallet), ECP TCI, Apple Meridian Bit Count

Below the settings, Sales and SET see the **Revision History**: every change, with date, user, and details.

Creating and editing (Sales)

1. On Configurations, click **New Configuration**. CALIGA shows the next free Config ID.
2. Enter a name and choose the settings. Fields left as "Not set" stay empty.
3. Click **Save Configuration**. It starts in POC, and the creation is logged in revision history.
4. While it's POC, change settings on the details page and click **Save Changes**. Each changed field is logged with its old and new value.

CALIGA checks values before saving: Facility Code must be a whole number, ECP TCI must be 6 hex characters, Meridian bit count must be 32, 40, 57, or 128, and BLE Advertising Name can have at most 5 characters. If anything is wrong, nothing is saved and a message says why.

Status actions (Sales)

- **Move to Active / Move to POC:** switches between the two. Active locks the settings and makes the profile testable.
- **Archive Configuration:** retires the profile after a confirmation. Tools and editing stop working.
- **Alert QA** (Sales and SET, Active profiles only): posts a test request in Slack for the firmware version picked in **Target FW**.

Action tools on the details page

Pick the firmware version in **Target FW** first. Every tool uses it.

Button	What happens
Download INI	Downloads the master INI if one exists. Otherwise CALIGA generates one from the database with INI-Generator. Review notes appear as comments at the top of a generated INI.
Download Profile Bin / Download Firmware Bin	Starts a background job. CALIGA returns an existing binary if one was already built, or asks Forge to build one and waits up to 60 seconds. The download starts when it's ready.
Flash Configuration (QA, SET)	Flashes the firmware to a reader plugged into your computer. See section 2.5.
Get Tokens	Shows the existing PocketBase tokens for this profile and firmware. Read-only.
Sync to PocketBase (inside Get Tokens)	Builds both binaries and uploads them to PocketBase, creating or replacing the records. Asks for confirmation.

2.3 Firmware testing (QA and SET)

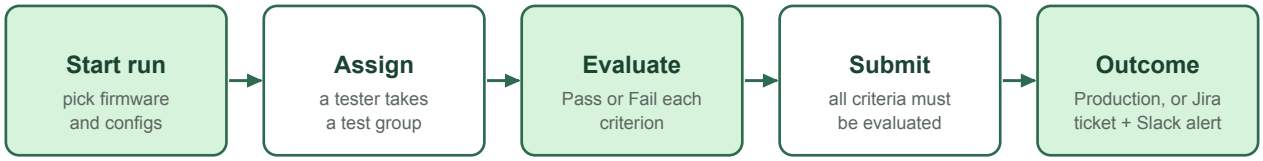


Figure 3. A test group from start to finish.

Starting a run

1. Open **Testing** and pick the firmware tab (v5.4.10 or v5.4.11).
2. Click **+ Start New Run**, choose the firmware version, and select the configurations to test. Only Active and Production configurations are offered. Production ones can be re-validated against new firmware.
3. Click **Initialize Test Run**. Each configuration becomes a test group with its own checklist.

Each checklist is built from the configuration's current settings:

Category	Criteria checked
Verify AV and Output Formats	Idle LED, Credential Report LED, Beeper, Keypad Format, Tamper Monitoring, Config ID
Present the HF credential	Leaf Si Application (Kv1), Leaf Cc Application (Kc1), Other Custom HF Application
Verify Card Serial Number (CSN) credentials	MFC, EV1/EV2, iClass, ISO 15693, ISO 14443A
Verify Low Frequency (LF) credentials	FSK Prox, ASK Prox, Prox Filter, Prox Filter Description
Verify Mobile configurations	BLE Advertising Config, BLE Functionality, NFC Functionality, MyPass Keyset, Legacy Credentials, Wallet (Apple, Android, both, or Off)

Testing a group

1. Open the group and click **Assign to Me**. Only the assigned tester can submit it.
2. Use **Flash Reader** on the group's row to load the firmware onto your reader if needed.
3. For each criterion, compare the reader with **Expected Configuration** and choose Passed or Failed. A failure needs a comment explaining what you saw.
4. Click **Submit Test Group Evaluation**. Every criterion must be Passed or Failed first.

What happens on submit:

- **All passed:** the group is Passed and the configuration moves to Production (unless it was archived meanwhile). The promotion is logged.
- **Any failed:** the group is Failed. CALIGA opens a Jira "Internal Bug" with the failed criteria and comments, moves it to "Next Up", and posts a failure alert in Slack. A Production configuration is demoted to its previous status, and the demotion is logged.
- A submitted group is locked. Run totals (passed and failed counts) are calculated from the groups, so they stay correct.

2.4 Failures (SET)

Failures lists every failed test group with its failed criteria, the tester's comments, and the Jira ticket.

Action	What it does
Claim Failure	Marks you as the owner and assigns the Jira ticket to you (when a Jira user's email matches yours exactly).
Unclaim	Removes your claim and unassigns the Jira ticket. You can only unclaim your own failures.
Inspect & Edit INI	Opens the INI. CALIGA loads your copy from `forge/jhepburn@wavelynx.com/<version>/` if you edited it before, otherwise the master INI.
Save INI	Saves your edited INI to the Forge folder in GCS. Nothing is built yet.
Flash Reader	Saves the edited INI, forces a fresh Forge build from it, and flashes the result to your reader. Stops at the first step that fails.
Re-Test	Resets every criterion to Untested, clears the tester and the claim, moves the Jira ticket to Complete, and posts a re-test request in Slack. The group shows up on Testing again.

2.5 Flashing a reader

Flashing runs in your browser through the RS-485 adapter plugged into your computer, so several people can flash their own readers from one CALIGA at the same time.

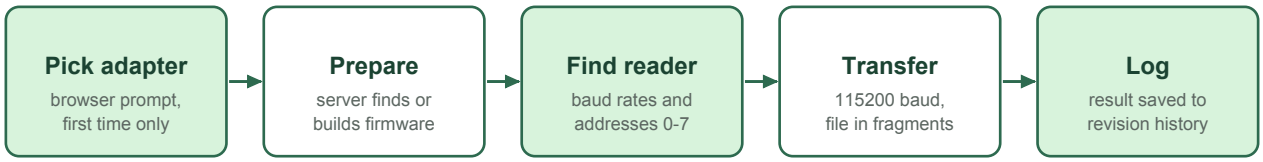


Figure 4. What happens when you click a Flash button.

Before you start: use Chrome or Edge, plug in the adapter, and power the reader. Close other programs that might be using the adapter.

1. Click **Flash Configuration** (details page) or **Flash Reader** (Testing or Failures).
2. The first time, the browser asks you to pick a serial port. Choose the adapter. The browser remembers it.
3. A progress window shows each step: preparing firmware, searching for the reader, raising the speed to 115200 baud, and the transfer percentage.
4. When it says the reader is rebooting, wait for the reboot to finish before testing.

Message	What to do
Flashing needs Chrome or Edge...	Open CALIGA in Chrome or Edge. Other browsers don't support Web Serial.
Could not open the adapter...	Another program (or another tab) is using the adapter. Close it and try again.
No reader answered at any standard baud rate...	Check wiring, power, and that the reader's OSDP address is 0 to 7.
The reader rejected the command (NAK...)	The reader refused a step. Power-cycle it and try again. If it repeats, the firmware file may not suit this reader.
Forge rejected the IAP cookie...	The Forge sign-in cookie expired. See section 3.6.
...could not be found or built within 60s	Forge didn't produce the binary in time. Try again, or check Forge.

2.6 Tools

Tools holds four engineering utilities. Every signed-in user can use them.

Partial Configuration Generator

Builds a small INI that overrides a few settings: anti-passback, BLE on or off, beeper, idle LED color, keypad bit format, OSDP address and baud rate, and CSN format per card technology. Choose the firmware version and whether to build a Profile BIN, then click **Generate**. CALIGA uploads the partial INI to your Forge folder (`partials/`), optionally builds the BIN (up to 120 seconds), and shows download links.

INI Translator

Upload any reader INI. CALIGA lists each section and setting with a plain-English description and a readable value (for example `true` becomes "Enabled / On"). Nothing is stored.

YAML Automator

Creates the YAML file used to program SAM chips. Pick the SAM type, partner, partner key, OEM, LEAF keys, TCI, ECP terminal settings, MiFare2Go application, and optional specialty keys. A new OEM name gets the next free SKU number, which is saved for next time. The file downloads as `SAM_Config_<OEM>_<timestamp>.yml`.

Batch Tokens

Select several non-archived configurations and a firmware version. **Look Up Tokens** reads existing tokens without changing anything. **Sync Selected** builds and uploads binaries for all of them (five at a time) and shows the tokens. Results appear as cards with copy buttons.

2.7 The progress window

Downloads of binaries, token syncs, the partial config tool, batch tokens, and flashing all run as background jobs. A progress window shows the current step. Closing the window doesn't stop a server job. A finished job's files stay available for a day. If CALIGA restarts during a job, the job is marked failed after 10 minutes and you can start it again.

Part 3. Setup and operations

3.1 What you need

Requirement	Why
Python 3.14	CALIGA runs on it. The project's virtual environment is `.venv`.
Access to the CALIGA PostgreSQL database	Its connection string goes in `DATABASE_URL`.
Google Cloud access to the `wavelynx_apex_config` bucket	Run `gcloud auth application-default login` once. CALIGA uses that sign-in to read and write GCS.
Credentials for Jira, Slack, Forge, and PocketBase	Each integration needs its own key in `.env` (section 3.3).
Chrome or Edge, and a USB RS-485 adapter	Only for flashing readers.
Node.js	Only for running the browser flasher's tests.

3.2 Installing

1. Clone the repository and open the folder in a terminal.
2. Create the virtual environment and install the pinned packages:

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
cp .env.example .env
```

3. Fill in `.env` (next section). `SECRET_KEY` and `DATABASE_URL` are required.
4. Create a secret key with `python3 -c "import secrets; print(secrets.token_hex(32))"`.
5. Bring the database up to date (section 3.4).
6. Start CALIGA with `.venv/bin/python app.py` and open `http://127.0.0.1:5000`.

Every package in `requirements.txt` is pinned to an exact version, including INI-Generator, which is pinned to a Git commit. To activate the environment in a terminal, run `source .venv/bin/activate`. In VS Code, choose `.venv/bin/python` as the interpreter.

3.3 Settings (.env)

CALIGA reads its settings from environment variables, loaded from `.env` at startup. Never commit `.env`. `.env.example` lists every setting without values.

Setting	Default	Purpose
`SECRET_KEY`	required	Signs sign-in sessions and form tokens. Keep it secret and stable.
`DATABASE_URL`	required	PostgreSQL connection, e.g. `postgresql+psycopg://USER:PASS@HOST:6543/DB`

Setting	Default	Purpose
FLASK_DEBUG	0	1 turns on the debugger and auto-reload. Local use only.
LOG_LEVEL	INFO	How much the server log shows.
AUTH_MODE	dev	How people sign in. Only dev (pick a user) exists today.
SESSION_HOURS	12	How long a sign-in lasts.
ALLOWED_EMAIL_DOMAIN	wavelynx.com	Only users with this email domain can be added or sign in.
SLACK_WEBHOOK_URL	none	Slack incoming webhook for alerts.
JIRA_DOMAIN	wavelynx.atlassian.net	Jira site.
JIRA_USER_EMAIL	none	Jira account CALIGA acts as.
JIRA_API_TOKEN	none	API token for that account.
JIRA_PROJECT_KEY	none	Project for failure tickets, e.g. SWAG. Without it, no tickets are created.
GCP_PROJECT_ID	erebus-257721	Google Cloud project for GCS.
GCS_BUCKET_NAME	wavelynx_apex_config	Bucket for master INIs and binaries. CALIGA writes only inside forge/jhepburn@wavelynx.com/.
FORGE_FOLDER_EMAIL	jhepburn@wavelynx.com	CALIGA writes to GCS only in forge/<this email>/, whoever is signed in.
GOOGLE_APPLICATION_CREDENTIALS	unset	Optional service-account key file. Leave unset to use your gcloud sign-in. If set to a missing file, CALIGA refuses to start.
FORGE_BASE_URL	https://forge.wavelynxdev.com	Forge address.
ACTIVE_IAP_COOKIE	none	Your Forge IAP cookie value (section 3.6). Without it, builds can't start.
ACTIVE_IAP_UID	built-in value	The IAP user ID cookie sent with the token.
FORGE_GCS_BUCKET	wavelynx_apex_config	Bucket Forge writes partial builds to.
PB_BASE_URL	https://unf.wavelynxtech.com	PocketBase address.
PB_COLLECTION_ID	cards	PocketBase collection that holds binaries.
PB_ADMIN_TOKEN	none	PocketBase admin token. It expires (section 3.6).
INI_GENERATOR_PRIVATE_DIR	required for INI generation	Local folder with INI-Generator's settings_default.ini and customer_profiles.json. Never stored in GCS.
JOB_WORKERS	2	Background worker threads per web process. 0 when a separate worker runs.

Setting	Default	Purpose
JOB_WORKER_THREADS	4	Threads for `flask --app app run-worker`.

3.4 Database and migrations

The database structure is managed with Flask-Migrate (Alembic). Each change is a numbered file in `migrations/versions/`. `flask --app app db upgrade` applies the ones the database hasn't had yet.

Migration	What it does
0001_baseline	The original tables, as they existed before migrations were added.
0002_review_fixes	Makes `config_id_dec` an integer (stops on non-numeric values), adds test case category and expected value, removes stored run totals, links test groups to configurations, drops the unused `failures` table, and creates `oem_skus`.
0003_ini_generator_fields	Adds Keyset ID, Custom Keyset, ECP DESFire, MiFare2Go, ECP TCI, Meridian bit count, and BLE Advertising Name.
0004_jobs	Creates the `jobs` table for the background queue.
0005_user_admin_active	Adds `is_admin` and `is_active` to users and makes jhepburn@wavelynx.com the admin.
0006_job_files	Creates `job_files`: background job output stored in the database instead of GCS.

Common commands

```
flask --app app db upgrade              # apply pending migrations
flask --app app db upgrade --sql        # print the SQL without running it
flask --app app db migrate -m "message" # create a migration after editing models.py
flask --app app db downgrade            # undo the latest migration
python init_db.py                       # add the team users to an empty database
```

New, empty database: run `db upgrade`, then `init_db.py`. **Database created before migrations existed:** run `flask --app app db stamp 0001_baseline` once, then `db upgrade`.

> **Important.** Back up the database and preview the SQL with `--sql` before upgrading a shared database. In zsh, don't add comments after a command: zsh passes them as arguments.

3.5 Running CALIGA

- Start: `.venv/bin/python app.py`. The site is at `http://127.0.0.1:5000`.
- The server log shows each request and what the integrations did, for example `[services.jira_service] Created Jira issue SWAG-137`.
- Background workers start with the first web request (two threads by default). For a separate worker, run `flask --app app run-worker` and set `JOB_WORKERS=0` on the web process.
- The database connection disables psycopg prepared statements. Supabase's transaction pooler (port 6543) shares connections between clients, and prepared statements would otherwise collide.

3.6 Routine maintenance

Task	How
Refresh the Forge IAP cookie	When builds fail with "Forge rejected the IAP cookie": open Forge in your browser, copy the value of the `__Host-GCP_IAP_AUTH_TOKEN_...` cookie into `ACTIVE_IAP_COOKIE` in `.env`, and restart CALIGA.
Refresh the PocketBase token	When Get Tokens says the token was rejected: get a new admin token from PocketBase, replace `PB_ADMIN_TOKEN`, and restart.
Update INI-Generator's private files	Upload the new `settings_default.ini` or `customer_profiles.json` in the folder named by `INI_GENERATOR_PRIVATE_DIR` and restart CALIGA.
Use a newer INI-Generator	Change the commit in the `ini-generator @ git+https://...@<commit>` line of `requirements.txt`, then run `pip install -r requirements.txt` and the tests.
Fill LEAF and wallet fields from master INIs	Run `python tools/backfill_ini_fields.py` to preview, then add `--apply`. It only fills blank fields and logs each change.
Add a firmware version	Add it to `SUPPORTED_FW_VERSIONS` in `utils/versions.py`, add its Forge build ID to `FIRMWARE_IDS` in `services/forge_service.py`, and add the option to the firmware dropdowns and tabs in the templates.
Add or remove people	Manage Users page (admin).

3.7 Tests

```
.venv/bin/pip install -r requirements-dev.txt
.venv/bin/python -m pytest
# To include the INI generation test, point it at INI-Generator's private files:
CALIGA_TEST_INI_PRIVATE_DIR=~/Desktop/INI-Generator/private .venv/bin/python -m pytest
```

Tests use a throwaway SQLite database. They blank every external credential and replace the service calls with stand-ins, so they never contact Slack, Jira, Forge, GCS, or PocketBase. The browser flasher test needs Node.js and skips itself without it. Section 4.11 lists every test file.

3.8 Troubleshooting

Symptom	Cause and fix
CALIGA won't start: "SECRET_KEY is not set" or "DATABASE_URL is not set"	Add the missing value to `.env`.
CALIGA won't start: "GOOGLE_APPLICATION_CREDENTIALS points to a missing file"	Fix the path, or remove the setting to use your gcloud sign-in.
CALIGA won't start: "AUTH_MODE ... isn't supported yet"	Set `AUTH_MODE=dev` until another mode is added.
Errors about missing columns or the `jobs` table	The database is behind. Run `flask --app app db upgrade`.
Builds fail with "Forge rejected the IAP cookie"	Refresh the cookie (section 3.6).
Tokens fail with "PocketBase rejected the admin token"	Refresh the token (section 3.6).
Download INI fails with "Can't generate INIs"	`INI_GENERATOR_PRIVATE_DIR` isn't set, or its folder is missing `settings_default.ini`. Point it at INI-Generator's private folder.
Progress window stays on "Queued..."	No worker is running. Workers start with the first web request unless `JOB_WORKERS=0`, in which case run `flask --app app run-worker`.
"Your form expired"	The form's security token didn't match. Reload the page and try again.
No Jira ticket on a failed test	Jira credentials or `JIRA_PROJECT_KEY` are missing. The failure is still recorded.
Flashing problems	See the table in section 2.5.

3.9 Before hosting CALIGA for others

CALIGA runs on one person's computer today. Sharing it on a server needs these steps first:

1. **Real sign-in.** Add a Google sign-in or Google IAP mode to `AUTH_MODE` and stop using `dev`. Either needs IT or DevOps: an OAuth client (Internal consent screen, redirect URI `https://<domain>/auth/callback`) or IAP in front of the site. The user list, roles, and blocking already work. The new mode only has to call `find_login_user` and `login_user` in `auth.py`.
2. **A production web server** such as gunicorn, in place of `python app.py`.
3. **HTTPS.** Browser flashing only works on HTTPS pages (or localhost).
4. **Secrets** stored in the hosting platform rather than an `.env` file.
5. **Workers.** Either keep a couple of worker threads per web process, or run a dedicated `run-worker` process and set `JOB_WORKERS=0`.
6. **A long-lived Forge credential.** The IAP cookie expires. A service account with an IAP token would avoid manual refreshes.

Part 4. Code guide

This part explains how the code is organized and what each file and important function does. Read 4.1 and 4.2 first: they show where everything lives and how a request travels through the application.

4.1 Repository layout

Path	Lines	Purpose
`app.py`	132	Builds the Flask application: settings, database, security, blueprints, error handling.
`auth.py`	96	Sign-in mode, current user, role and admin checks, the unknown-user block.
`models.py`	264	Database tables as SQLAlchemy models, plus small helper properties.
`jobs.py`	242	The shared background job queue: enqueue, workers, progress, files, cleanup.
`routes/`	1,449	One blueprint per area: main, configurations, testing, failures, tools, jobs, admin, plus helpers.
`services/`	1,412	Integrations and file handling: GCS (with the forge-folder write guard), Forge, INI generation and translation, Jira, Slack, PocketBase.
`templates/`	2,713	Jinja HTML pages. `base.html` holds the layout and the shared JavaScript.
`static/js/osdp-flasher.js`	353	Browser OSDP flashing over Web Serial.
`utils/`	24	Firmware version, CSN code, and error helpers.
`tools/backfill_ini_fields.py`	141	One-off script that fills LEAF and wallet fields from master INIs.
`migrations/`	511	Alembic environment and the six migrations.
`tests/`	1,320	pytest suite, Node test and reference frames for the flasher, and support scripts.
`init_db.py`	33	Adds the team users to an empty database.
`requirements.txt`, `requirements-dev.txt`	18	Pinned runtime and test packages.
`README.md`, `.env.example`	217	Quick-start documentation and the settings template.

4.2 How a request flows

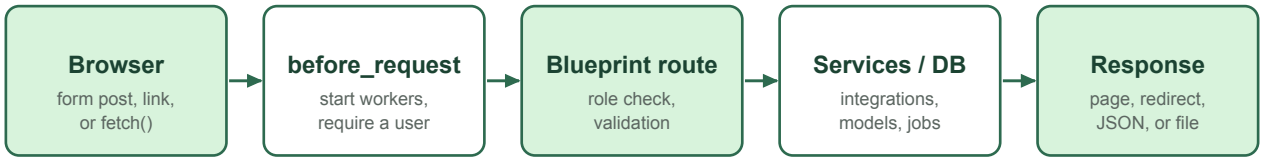


Figure 5. The path of every request.

`app.py` builds the application once, when the module is imported:

1. Loads `.env` before importing anything else. The services read their settings when they're imported, so the order matters.
2. Sets up logging (`LOG_LEVEL`), and stops with a clear error if `SECRET_KEY` or `DATABASE_URL` is missing or the Google credentials file doesn't exist. Logs a warning in development login mode.
3. Configures Flask: the session lifetime, CSRF tokens without a time limit (flashes and builds can take a while), and database pool options. For psycopg it disables prepared statements, because Supabase's transaction pooler would make them collide.
4. Connects SQLAlchemy, Flask-Migrate, and CSRF protection, then registers the seven blueprints.
5. Adds a **context processor** that gives every template `current_user`, `dev_login`, and (in development login) the list of active users for the menu.
6. Adds two **before_request** hooks. The first starts the job workers once. The second sends anyone without a signed-in user to Home (or answers 401 to JSON requests), except on the public pages: Home, user switching, sign out, and static files.
7. Adds **error handlers**. A CSRF failure shows "Your form expired". A 403 shows the reason as a message and returns to the previous page (or Home for page loads). 403 and 404 answer with JSON when the browser asked for JSON.
8. Adds the `run-worker` command for a dedicated worker process.

Pages and JavaScript tell CALIGA they want JSON by sending `Accept: application/json`. `wants_json()` in `routes/helpers.py` checks this, so the same route can serve a form post (redirect plus message) and a fetch call (JSON).

4.3 Sign-in and permissions (auth.py)

Name	What it does
`auth_mode()`	Reads `AUTH_MODE`. Only `dev` is supported. Any other value stops the app, so a typo can't quietly let everyone in.
`find_login_user(email)`	Returns the active user with that email. Raises `LoginBlocked` for other domains, unknown users, and deactivated users. Every future sign-in method uses it.
`login_user(user)` / `logout_user()`	Start or end a session. Login clears the old session first and marks it permanent so `SESSION_HOURS` applies.
`get_current_user()`	Loads the session's user once per request (cached in `flask.g`). A deactivated user counts as signed out.
`@require_role(*roles)`	Decorator. Answers 403 "This action requires the X role" unless the user has one of the roles.
`@require_admin`	Decorator. Answers 403 unless the user is an admin.

The development login lives in `routes/main.py` as `/switch_user/<role>?email=...`. It only works when `AUTH_MODE=dev` and only for active users.

4.4 Data model (models.py)

Each class is a database table. Section 5.3 lists every column. Timestamps are stored as UTC without a time zone, created by `utcnow()`.

Model (table)	What it holds	Helpers
`User` (users)	Name, email, role, `is_admin`, `is_active`.	`display_name` ("James H.")
`Configuration` (configurations)	Every reader setting, status, previous status, IDs, notes, timestamps. Includes older imported columns (`et10_ws_2` and similar) that CALIGA doesn't edit.	`leaf_si_enabled`, `leaf_cc_enabled`: On/Off fields, falling back to Card Type.
`RevisionHistory` (revision_history)	One row per change to a configuration, linked by name.	
`FWRun` (fw_runs)	A firmware run: version and date.	`passed_total`, `failed_total`: counted from the groups.
`ConfigurationTestGroup` (configuration_test_groups)	One configuration in a run: status, tester, claim, Jira key.	`cases_by_category`: criteria grouped in their original order.
`TestCase` (test_cases)	One criterion: category, name, expected value, status, comment.	`display_category`, `display_label`, `display_expected`: also read rows saved before the columns existed.
`OemSku` (oem_skus)	OEM names and SKU numbers for the YAML automator.	
`Job` (jobs)	Background jobs: kind, owner, status, progress, parameters, result, files, worker, timestamps.	`to_dict()` for the status API.

> **Note.** Config IDs aren't unique: CBA6 and CAM3 both use 8, on purpose. New IDs are handed out under a database lock, so two people creating configurations at once never get the same number.

4.5 Routes (routes/)

Section 5.1 lists all 40 routes with their methods and required roles. This section explains the logic behind them.

routes/helpers.py

- `get_usable_config(name)`: loads a configuration, answering 404 if it doesn't exist and 403 if it's Archived. Every download, build, token, and flash route uses it.
- `log_revision(name, details)`: adds a revision history row for the current (or given) user.
- `set_status(config, new)`: changes status and records the old one, only when it actually changes. This keeps `previous_status` correct when a Production configuration passes again.
- `send_and_cleanup(path, name, dir)`: reads a file into memory, deletes its temporary folder, and sends it.

routes/configurations.py

- `EDITABLE_FIELDS` lists the fields the forms may change. `_parse_field` validates each one (facility code, Meridian bit count, TCI, BLE name) and normalizes TCI and BLE name to uppercase.
- `update_configuration` compares old and new values, saves only changes, and logs them as "Field: 'old' to 'new'". One invalid value cancels the whole save.
- `create_configuration` takes a Postgres advisory lock (`pg_advisory_xact_lock`), computes the next ID as the highest plus one, and commits. The lock is released at commit.
- `toggle_status` switches POC and Active only. `archive_configuration` archives anything but Archived. `alert_qa` requires Active and posts to Slack.

routes/testing.py

- `build_test_criteria(config)` returns the checklist as (category, label, expected value) rows, from the configuration's current settings.
- `start_new_run` accepts only Active and Production configurations, warns about any it skipped, and creates the run, test groups, and test cases in one commit.
- `submit_test_group` enforces the rules in order: not already submitted, a tester assigned, the submitter is the tester, comments on failures, nothing left Untested. On failure it creates the Jira ticket, sends the Slack alert, and demotes a Production configuration to its previous status (Active if that's unclear). On pass it promotes to Production. Both are logged.

routes/failures.py

- `failures_dashboard` lists Failed groups. INIs aren't loaded here. `failure_ini` returns one INI as JSON when its drawer opens, so the page loads quickly.
- `_failure_firmware_job` (job kind `failure_firmware`) saves the edited INI, then calls `get_or_build_firmware_bin(..., force_rebuild=True)` so the firmware comes from the edited INI, not an old build.
- `claim_failure`, `unclaim_failure`, and `retest_failure` update the group and call Jira and Slack. Missing Jira keys produce a clear message instead of touching an unrelated ticket.

routes/tools.py

- Download, token, and flash routes for one configuration, plus the four tools.
- Slow work runs as jobs. Each handler is registered with `@job_handler(kind)`: `binary` (profile or firmware), `sync_tokens`, `partial`, `batch_lookup`, and `batch_sync`. Routes call `_start(kind, **params)`, which queues the job and returns its ID with status 202.
- `flash_log` records a browser flash result in revision history (QA and SET only).
- The YAML automator builds a Python dictionary and writes it with PyYAML. `HexInt` makes chosen numbers print as hex (`0xC3`). New OEM names get the next SKU number in `oem_skus`.

routes/jobs.py, routes/admin.py, routes/main.py

- `/jobs/<id>` returns a job's status as JSON. `/jobs/<id>/files/<key>` streams one of its files. Both answer 404 for jobs that belong to someone else.
- The admin routes list users, add users (validated name, @wavelynx.com email, role, no duplicates), and change a role or active flag. An admin can't deactivate their own account.
- `main` has Home, the development login, and sign out.

4.6 Background job queue (jobs.py)



Figure 6. A job's life.

Piece	Behavior
`enqueue(kind, owner_id, **params)`	Inserts a `jobs` row with JSON parameters and wakes local workers.
`_claim_next(app)`	Selects the oldest pending job with `FOR UPDATE SKIP LOCKED`, then marks it running only if it's still pending. Two workers, even in different processes, can never take the same job.
`JobContext`	What a handler receives: a temporary `workdir`, `result` (a dictionary saved at the end), and `message` / `progress` properties that write straight to the database on their own connection (progress at most once a second). `add_file(key, path, name)` saves a file in the `job_files` table.
`_run(app, job_id)`	Runs the handler with a heartbeat thread (every 30 s). `UserFacingError` messages are shown as-is. Other errors are logged and shown as "Unexpected error". The temporary folder is always deleted.
`_reap(app)`	Every minute: running jobs without a heartbeat for 10 minutes become failed. Finished jobs older than a day are deleted with their files.
`start_workers(app, count)`	Starts the worker threads once per process. Called by the first request or by `run-worker`.
`job_files` table	Holds each job's output files. Jobs never write to GCS. Rows are deleted with their job.

To add a new kind of job: write `def handler(job, **params)` decorated with `@job_handler("name")` in a route module, call `enqueue("name", user_id, ...)` with JSON-safe parameters, and use `CALIGA.runJob` or `CALIGA.downloadJob` in the page.

4.7 Services (services/)

GCS: gcs_client.py and gcs_service.py

`get_gcs_client()` creates one shared client. `gcs_service.py` knows the bucket layout. CALIGA reads from all of these paths but writes only inside `forge/jhepburn@wavelynx.com/`:

Bucket path	Contents
`input/<version>/<config>.ini`	Master INIs.
`output/<version>/<timestamp folder>/`	Official build output.
`forge/jhepburn@wavelynx.com/<version>/`	The shared Forge workspace: generated and edited INIs, and Forge's builds of them.
`forge/jhepburn@wavelynx.com/<version>/partials/`	Partial INIs and their builds.

- `find_latest_matching_blob` searches a prefix (and its subfolders) for exact file names and returns the newest. It uses GCS `match_glob` so the server filters, not CALIGA.
- `get_or_create_ini_file`: master INI (version with and without "v"), then your Forge copy, otherwise generate from the database and upload a copy to the Forge folder.

- `_get_or_build_binary` (used by `get_or_build_profile_bin` and `get_or_build_firmware_bin`): looks in `output/` then `forge/`. If nothing is found, it makes sure an INI exists, starts a Forge build, and checks `forge/` every 3 seconds for up to 60. With `force_rebuild=True` it skips existing files and only accepts a file written after the build started.
- `get_failure_ini_content` and `save_edited_failure_ini` read and write the Failures page's INI.
- `write_to_forge(bucket, path, email)` in `gcs_client.py` is the only way CALIGA writes to GCS. It refuses any path outside `forge/jhepburn@wavelynx.com/` (set by `FORGE_FOLDER_EMAIL`), whoever is signed in. A test fails if any other code uploads directly.

Forge: forge_service.py

- `trigger_forge_build` posts to `/configs/build` with build type firmware, the firmware build ID from `FIRMWARE_IDS` (v5.4.10: 371132, v5.4.11: 422313), source user, and Git ref master.
- `compile_partial_bin_via_forge` starts a profile build with partials, then watches `partials/` for up to 120 seconds for a new `.bin` or `.dck`.
- `_check_iap_auth` treats 401, 403, or a redirect to Google sign-in as an expired cookie and raises `ForgeAuthError` with instructions. The job fails straight away with that message.
- `generate_partial_ini_content` turns the partial tool's form into INI lines (see the mapping in section 2.6).

INI generation: ini_generator.py

CALIGA doesn't contain INI rules itself. It installs the INI-Generator package and feeds it the configuration as form rows, the same layout as the original configuration spreadsheet:

```
[["Config Name", "", "CQU1"], ["Idle LED", "", "Red"], ["MFC CSN", "", "On", "32-bit msb"], ...]
```

- `config_to_rows(config)` builds the rows. CSN values are translated ("Standard" becomes `0x2001`, "56-bit LSB" becomes `0x3800`, and so on). Unknown values pass through so INI-Generator can flag them.
- `_generator()` imports INI-Generator once, pointed at the private files in `INI_GENERATOR_PRIVATE_DIR`. INI-Generator reads them when it's imported. They're never read from or written to GCS.
- `generate_ini_from_config(config, path)` writes the INI with a comment header listing anything that needs manual review, and returns those warnings.

INI translation, Jira, Slack, PocketBase

File	What it does
`ini_translator_service.py`	Parses an uploaded INI and describes each setting using two dictionaries: 91 setting descriptions and 24 section titles. Adds a `[GENERAL]` section to files that have none.
`jira_service.py`	Creates "Internal Bug" tickets titled "Test Failure: CONFIG on FW VERSION" and moves them to Next Up. Assigns only on an exact email match. Unassigns with a null assignee. Moves tickets to Complete, Done, Resolved, or Closed.
`slack_service.py`	Two webhook messages: "QA Test Request Triggered" (used for Alert QA and re-tests) and "Test Suite Failure Detected".
`pocketbase_service.py`	Record names are "5.4.10 CQU1 PROFILE" and "5.4.10 CQU1 FIRMWARE". `lookup_tokens` reads them. `sync_tokens` builds both binaries and updates or creates the records (new tokens look like `c–XXXXXXXX`). A rejected token produces a message saying it probably expired.

4.8 Pages and shared JavaScript (templates/)

Template	Page
`base.html`	Layout, top bar, user menu, messages, development banner, progress window, shared JavaScript.
`home.html`	Role and status descriptions.
`configurations.html`	Configuration list with search and status filter.
`configuration_details.html`	Details, editing, status actions, tools, tokens, revision history.
`create_configuration.html`	New configuration form.
`_ini_fields.html`	Macros for the LEAF & Wallet card and BLE name row, shared by create and details.
`testing.html`	Firmware tabs, runs, test groups, start-run window.
`failures.html`	Failures with claim, INI drawer, Flash Reader, Re-Test.
`tools.html` and `tools/*.html`	Tools list, partial config, INI translator, YAML automator, batch tokens.
`admin_users.html`	Manage Users.

Every form includes a hidden `csrf_token`. `base.html` defines a `CALIGA` object used by every page:

Function	Use
`CALIGA.post(url, data)` / `getJson(url)`	fetch calls that send the CSRF token and ask for JSON. Errors become readable messages.
`CALIGA.pollJob(id, onUpdate)`	Checks a job every 1.5 seconds until it's done or failed.
`CALIGA.runJob({title, url, data, onDone})`	Starts a job and shows the progress window.
`CALIGA.downloadJob(...)`	Runs a job and downloads its file when it's done.
`CALIGA.flashReader({prepareUrl, prepareData, logUrl})`	The whole browser flash: pick port, prepare firmware, flash, log.
`CALIGA.el(...)`, `tokenCard(...)`	Build page elements with text (never raw HTML), so server data can't inject markup.

Two page-wide handlers save repeated code: forms with `data-confirm` ask before submitting, and any element with `data-flash-prepare-url` starts a flash.

4.9 Browser flasher (static/js/osdp-flasher.js)

The flasher speaks OSDP exactly the way CALIGA's original Python OSDP library did, which flashed from the server before flashing moved to the browser. The tests compare its output with frames recorded from that library, byte for byte. It works with any "transport" that can write, read with a timeout, change baud rate, and clear input. `WebSerialTransport` is the browser one.

Command frame (each one is sent after a `0xFF` line-mark byte):

Byte(s)	Field	Notes
0	Start of message	`0x53`
1	Address	0 to 126. Replies set the top bit (`0x80`).
2-3	Length	Whole frame, little-endian.
4	Control	Sequence number (bits 0-1) and the CRC flag (`0x04`). Secure channel isn't used.
5	Command	POLL `0x60`, ID `0x61`, COMSET `0x6E`, FILETRANSFER `0x7C`.
6...	Data	COMSET: address + baud (4 bytes). FILETRANSFER: type 1, total size (4), offset (4), fragment size (2), fragment.
last 2	CRC	CRC-16/CCITT, initial value `0x1D0F`, little-endian.

- Sequence numbers go 0, 1, 2, 3, 1, 2, 3... A reply with sequence 0 means the reader reset and the count starts again.
- `discoverReader` tries 115200 first, then 9600, 19200, 38400, 57600, polling addresses 0 to 7 with a 300 ms timeout each. A silent address doesn't stop the scan.
- `flashFirmware` raises the reader to 115200 with COMSET if needed, checks ID and POLL, then sends 1,024-byte fragments. It follows the reader's FTSTAT replies: a requested fragment size, a requested delay (up to 5 s), FINISHING (sends empty fragments), and REBOOTING (done). A reader that goes silent right after the last fragment counts as rebooting. NAKs and negative status codes stop the flash

with a readable message. Finally it tries to set the original baud rate back.

4.10 Utilities, scripts, and migrations

File	What it does
`utils/versions.py`	`SUPPORTED_FW_VERSIONS` (v5.4.10, v5.4.11), `normalize_fw_version` ("v5.4.10" and "5.4.10"), `parse_fw_version` (falls back to v5.4.10 for anything unknown, which keeps GCS paths safe).
`utils/csn.py`	CSN codes for the partial tool: Off `0xFFFF`, no override `0x0000`, 32-bit MSB `0x2001`, 56-bit LSB `0x3800`, 56-bit MSB `0x3801`.
`utils/errors.py`	`UserFacingError`: the base for errors whose message is safe to show (Forge, INI generation, jobs, sign-in).
`tools/backfill_ini_fields.py`	Reads master INIs and fills blank LEAF, keyset, wallet, and BLE name fields. Ignores default values such as `config_card` and the "APEX" name. Dry run by default.
`init_db.py`	Adds the eight team users to an empty database, with James H. as admin.
`migrations/env.py`	Standard Flask-Migrate environment. Uses the app's database settings.

4.11 Tests (tests/)

File	Covers
`conftest.py`	Test setup: SQLite database, blank credentials, five test users (one admin), helpers to sign in, create configurations, and wait for jobs.
`test_auth_and_pages.py`	Signing in, role gates, CSRF, every page for every role.
`test_configurations.py`	Create, edit, toggle, archive, alert QA, search, LEAF and wallet validation.
`test_testing_workflow.py`	Runs, submit rules, promotion, demotion, totals.
`test_failures.py`	Claim, re-test, INI loading, edited-INI firmware builds.
`test_tools_and_jobs.py`	Binary jobs, job ownership, partial tool, tokens, flash logging, YAML automator.
`test_forge_and_csn.py`	IAP cookie detection and the partial tool's CSN output.
`test_ini_generation.py`	INI-Generator form rows, backfill parsing, generated INI contents (needs private files).
`test_users_admin.py`	Sign-in rules, Manage Users, deactivation, sign out.
`test_job_queue_processes.py`	Jobs queued in one process run exactly once in two other processes.
`test_osdp_flasher_js.py`	Runs `tests/js/osdp-flasher.test.mjs`: frames match the recorded reference frames in `tests/js/osdp_reference.json` byte for byte, and simulated readers cover discovery, baud change, fragment sizes, endings, NAKs, and no reader.

4.12 Conventions to keep

- **Check permissions on the server.** Every action that changes data uses `@require_role` or `@require_admin`.
- **Log configuration changes.** Use `log_revision` whenever a configuration's settings or status change.
- **Show users real reasons.** Raise `UserFacingError` (or a subclass) with a message a person can act on.
- **Don't invent fallbacks.** No placeholder emails, Jira keys, or tokens. If something is missing, say so.
- **Pin every package** to an exact version, and pin Git dependencies to a commit.
- **Use** `logging`, not `print`.
- **Add or update tests** with every feature.

4.13 Known limitations and future work

Item	Status
Google sign-in or IAP	Waiting on IT or DevOps. Everything else about users and roles is ready (section 3.9).
Hosting	Not set up. See section 3.9.
Forced rebuilds for downloads	Only the Failures flash forces a fresh build. Downloads can return an older binary. To be discussed.
Planned features	Wallet credentials (single and batch) and tagging assigned users in Slack.

Part 5. Reference

5.1 Routes

"Signed in" means any active user. Every POST form also needs the page's CSRF token.

Path	Method	Access	Purpose
`/`	GET	Public	Home: role and status descriptions.
`/switch_user/<role>`	GET	Public (dev login only)	Act as an active user. Optional ?email= picks one person.
`/logout`	POST	Public	Sign out.
`/admin/users`	GET	Admin	Manage Users page.
`/admin/users`	POST	Admin	Add a user.
`/admin/users/<user_id>`	POST	Admin	Change a user's role or active flag.
`/configurations`	GET	Signed in	List and search configurations.
`/configurations/<config_name>`	GET	Signed in	Configuration details.
`/configurations/new`	GET	Sales	New configuration form.
`/configurations/create`	POST	Sales	Create a configuration in POC.
`/configurations/<config_name>/update`	POST	Sales	Save changes to a POC configuration.
`/configurations/toggle_status/<config_name>`	POST	Sales	Switch between POC and Active.
`/configurations/archive/<config_name>`	POST	Sales	Archive a configuration.
`/configurations/<config_name>/alert-qa`	POST	Sales, SET	Slack test request for an Active configuration.
`/configurations/<config_name>/download-ini`	GET	Signed in	Download the master or generated INI.
`/configurations/<config_name>/download-profile-bin`	POST	Signed in	Job: find or build the profile binary.
`/configurations/<config_name>/download-firmware-bin`	POST	Signed in	Job: find or build the firmware binary. Also prepares firmware for flashing.
`/configurations/<config_name>/flash-log`	POST	QA, SET	Record a browser flash result in revision history.
`/configurations/<config_name>/get-tokens`	GET	Signed in	Read existing PocketBase tokens.
`/configurations/<config_name>/sync-tokens`	POST	Signed in	Job: build both binaries and upload them to PocketBase.
`/testing`	GET	Signed in	Firmware runs for a version (?fw_version=).

Path	Method	Access	Purpose
`/testing/start-run`	POST	QA, SET	Start a firmware run.
`/testing/assign-group/<group_id>`	POST	QA, SET	Assign or unassign yourself.
`/testing/submit-group/<group_id>`	POST	QA, SET (assigned tester)	Submit results.
`/failures`	GET	SET	Failures dashboard.
`/failures/ini/<group_id>`	GET	SET	INI for the edit drawer (JSON).
`/failures/save-ini/<group_id>`	POST	SET	Save the edited INI to the Forge folder.
`/failures/flash_and_run/<group_id>`	POST	SET	Job: save the INI and force a firmware build for flashing.
`/failures/claim/<group_id>`	POST	SET	Claim and assign the Jira ticket.
`/failures/unclaim/<group_id>`	POST	SET (owner)	Unclaim and unassign the Jira ticket.
`/failures/retest/<group_id>`	POST	SET	Reset the suite, close the Jira ticket, alert QA.
`/tools`	GET	Signed in	Tools list.
`/tools/partial-config`	GET, POST	Signed in	Partial config form, and the job that builds it.
`/tools/ini-translator`	GET, POST	Signed in	Upload and translate an INI.
`/tools/yaml-automator`	GET, POST	Signed in	SAM chip YAML form and download.
`/tools/batch-tokens`	GET	Signed in	Batch tokens page.
`/tools/batch-tokens/lookup`	POST	Signed in	Job: read tokens for several configurations.
`/tools/batch-tokens/sync`	POST	Signed in	Job: build and upload for several configurations.
`/jobs/<job_id>`	GET	Job owner	Job status (JSON).
`/jobs/<job_id>/files/<key>`	GET	Job owner	Download a job's file.

5.2 Pinned packages

Package	Version	Use
Flask	3.0.3	Web framework
Flask-SQLAlchemy	3.1.1	Database models
Flask-Migrate	4.1.0	Migrations
alembic	1.20.0	Migration engine
Flask-WTF	1.2.2	CSRF protection
SQLAlchemy	2.0.51	Database toolkit
psycopg / psycopg-binary	3.3.6	PostgreSQL driver
python-dotenv	1.0.1	Loads .env
requests	2.33.1	HTTP calls to Forge, Jira, Slack, PocketBase
google-cloud-storage	2.15.0	GCS
PyYAML	6.0.3	YAML automator
Jinja2	3.1.6	Templates
Werkzeug	3.1.8	Flask's server toolkit
ini-generator	commit ae5087d	INI rules (github.com/jhepburn-hue/INI-Generator)
cryptography	46.0.7	Needed by google-auth, which Google Cloud Storage uses to sign in
pytest (dev)	8.3.4	Tests

5.3 Database tables

users

People who can use CALIGA.

Column	Type	Null	Notes
`id`	INTEGER	no	primary key
`first_name`	VARCHAR(100)	no	
`last_name_initial`	VARCHAR(5)	no	
`email`	VARCHAR(150)	no	unique
`role`	VARCHAR(50)	no	
`is_admin`	BOOLEAN	no	
`is_active`	BOOLEAN	no	

configurations

One row per reader configuration profile.

Column	Type	Null	Notes
id	INTEGER	no	primary key
config_name	VARCHAR(100)	no	unique
config_id_hex	TEXT	yes	
config_id_dec	INTEGER	yes	indexed
status	VARCHAR(50)	no	
previous_status	VARCHAR(50)	yes	
et10_ws_2	TEXT	yes	
et10_ws_3	TEXT	yes	
et10_ws_6	TEXT	yes	
et10_ws_7	TEXT	yes	
et20_ws_2	TEXT	yes	
et20_ws_3	TEXT	yes	
et20_ws_6	TEXT	yes	
et20_ws_7	TEXT	yes	
et25_ws_2	TEXT	yes	
et25_ws_3	TEXT	yes	
et25_ws_6	TEXT	yes	
et25_ws_7	TEXT	yes	
et10_f2f	TEXT	yes	
et20_f2f	TEXT	yes	
et25_f2f	TEXT	yes	
et10_mclp	TEXT	yes	
et20_mclp	TEXT	yes	
et25_mclp	TEXT	yes	
idle_led	TEXT	yes	
credential_report_led	TEXT	yes	
beeper	TEXT	yes	
keypad_format	TEXT	yes	
tamper_monitoring	TEXT	yes	
casi_output_format	TEXT	yes	
supervision_state	TEXT	yes	
tamper_monitoring_f2f	TEXT	yes	

Column	Type	Null	Notes
leaf_si	TEXT	yes	
leaf_cc	TEXT	yes	
other_custom_hf_application	TEXT	yes	
custom_application_notes	TEXT	yes	
mfc_csn	TEXT	yes	
ev1_ev2_csn	TEXT	yes	
iclass_csn	TEXT	yes	
iso_15693_csn	TEXT	yes	
iso_14443a_csn	TEXT	yes	
fsk_prox	TEXT	yes	
ask_prox	TEXT	yes	
prox_filter	TEXT	yes	
prox_filter_description	TEXT	yes	
ble_advertising_config	VARCHAR(100)	yes	
ble_functionality	VARCHAR(100)	yes	
nfc_functionality	TEXT	yes	
mobile_keyset	TEXT	yes	
legacy_credentials	TEXT	yes	
transport_mode	TEXT	yes	
mobile_notes	TEXT	yes	
keyset_id	TEXT	yes	
custom_keyset	TEXT	yes	
ecp_desfire	TEXT	yes	
mifare_2go	TEXT	yes	
ecp_tci	TEXT	yes	
meridian_bit_count	INTEGER	yes	
ble_advertising_name	VARCHAR(20)	yes	
assigned_part_numbers	TEXT	yes	
card_type	TEXT	yes	
bitstream	TEXT	yes	
facility_code	INTEGER	yes	
starting_badge	TEXT	yes	
card_notes	TEXT	yes	
card_part_numbers	TEXT	yes	

Column	Type	Null	Notes
notes_other_products	TEXT	yes	
created_at	DATETIME	yes	
updated_at	DATETIME	yes	

revision_history

Change log for configurations.

Column	Type	Null	Notes
id	INTEGER	no	primary key
config_name	VARCHAR(100)	no	FK to configurations.config_name
revision_date	DATETIME	no	
user_email	VARCHAR(150)	no	
revision_details	TEXT	no	

fw_runs

Firmware test runs.

Column	Type	Null	Notes
id	INTEGER	no	primary key
fw_version	VARCHAR(20)	no	
run_date	DATETIME	yes	

configuration_test_groups

One configuration within a run.

Column	Type	Null	Notes
id	INTEGER	no	primary key
fw_run_id	INTEGER	no	FK to fw_runs.id
configuration_id	INTEGER	yes	FK to configurations.id
config_name	VARCHAR(100)	no	
assigned_user	VARCHAR(120)	yes	
claimed_user	VARCHAR(120)	yes	
status	VARCHAR(20)	yes	
jira_key	VARCHAR(50)	yes	

test_cases

One criterion within a test group.

Column	Type	Null	Notes
id	INTEGER	no	primary key
test_group_id	INTEGER	no	FK to configuration_test_groups.id
category	VARCHAR(200)	yes	
criterion_name	VARCHAR(200)	no	
expected_value	TEXT	yes	
status	VARCHAR(20)	yes	
comment	TEXT	yes	

oem_skus

OEM SKU numbers for the YAML automator.

Column	Type	Null	Notes
id	INTEGER	no	primary key
name	VARCHAR(100)	no	unique
sku_id	INTEGER	no	

jobs

Background jobs.

Column	Type	Null	Notes
id	VARCHAR(32)	no	primary key
kind	VARCHAR(50)	no	
owner_id	INTEGER	no	FK to users.id
status	VARCHAR(20)	no	indexed
message	TEXT	yes	
progress	FLOAT	yes	
params	JSON	no	
result	JSON	yes	
files	JSON	yes	
error	TEXT	yes	
worker	VARCHAR(100)	yes	
created_at	DATETIME	no	indexed
started_at	DATETIME	yes	

Column	Type	Null	Notes
heartbeat_at	DATETIME	yes	
finished_at	DATETIME	yes	

job_files

Files produced by background jobs. Deleted with their job.

Column	Type	Null	Notes
id	INTEGER	no	primary key
job_id	VARCHAR(32)	no	FK to jobs.id, indexed
key	VARCHAR(50)	no	
name	VARCHAR(255)	no	
data	BLOB	no	
created_at	DATETIME	no	